

```
void elsolve(VecDoub_I &b, VecDoub_O &y) {
```

Solve $\mathbf{L} \cdot \mathbf{y} = \mathbf{b}$, where $\mathbf{L}$ is the lower triangular matrix in the stored Cholesky decomposition. $\mathbf{b}[0..n-1]$ is input as the right-hand side vector. The solution vector is returned in $\mathbf{y}[0..n-1]$.

```
    Int i,j;
    Doub sum;
    if (b.size() != n || y.size() != n) throw("bad lengths");
    for (i=0;i<n;i++) {
        for (sum=b[i],j=0; j<i; j++) sum -= el[i][j]*y[j];
        y[i] = sum/el[i][i];
    }
}
```

```
void inverse(MatDoub_O &ainv) {
```

Set $\mathbf{ainv}[0..n-1][0..n-1]$ to the matrix inverse of $\mathbf{A}$, the matrix whose Cholesky decomposition has been stored.

```
    Int i,j,k;
    Doub sum;
    ainv.resize(n,n);
    for (i=0;i<n;i++) for (j=0;j<=i;j++){
        sum = (i==j? 1. : 0.);
        for (k=i-1;k>=j;k--) sum -= el[i][k]*ainv[j][k];
        ainv[j][i] = sum/el[i][i];
    }
    for (i=n-1;i>=0;i--) for (j=0;j<=i;j++){
        sum = (i<j? 0. : ainv[j][i]);
        for (k=i+1;k<n;k++) sum -= el[k][i]*ainv[j][k];
        ainv[i][j] = ainv[j][i] = sum/el[i][i];
    }
}
```

```
Doub logdet() {
```

Return the logarithm of the determinant of $\mathbf{A}$, the matrix whose Cholesky decomposition has been stored.

```
    Doub sum = 0.;
    for (Int i=0; i<n; i++) sum += log(el[i][i]);
    return 2.*sum;
}
```

```
};
```

CITED REFERENCES AND FURTHER READING:

- Wilkinson, J.H., and Reinsch, C. 1971, *Linear Algebra*, vol. II of *Handbook for Automatic Computation* (New York: Springer), Chapter I/1.
- Gill, P.E., Murray, W., and Wright, M.H. 1991, *Numerical Linear Algebra and Optimization*, vol. 1 (Redwood City, CA: Addison-Wesley), §4.9.2.
- Dahlquist, G., and Björck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §5.3.5.
- Golub, G.H., and Van Loan, C.F. 1996, *Matrix Computations*, 3rd ed. (Baltimore: Johns Hopkins University Press), §4.2.

2.10 QR Decomposition

There is another matrix factorization that is sometimes very useful, the so-called *QR decomposition*,

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \quad (2.10.1)$$

Here $\mathbf{R}$ is upper triangular, while $\mathbf{Q}$ is orthogonal, that is,

$$\mathbf{Q}^T \cdot \mathbf{Q} = \mathbf{1} \quad (2.10.2)$$

where $\mathbf{Q}^T$ is the transpose matrix of $\mathbf{Q}$. Although the decomposition exists for a general rectangular matrix, we shall restrict our treatment to the case when all the matrices are square, with dimensions $N \times N$.

Like the other matrix factorizations we have met (LU , SVD, Cholesky), QR decomposition can be used to solve systems of linear equations. To solve

$$\mathbf{A} \cdot \mathbf{x} = \mathbf{b} \quad (2.10.3)$$

first form $\mathbf{Q}^T \cdot \mathbf{b}$ and then solve

$$\mathbf{R} \cdot \mathbf{x} = \mathbf{Q}^T \cdot \mathbf{b} \quad (2.10.4)$$

by backsubstitution. Since QR decomposition involves about twice as many operations as LU decomposition, it is not used for typical systems of linear equations. However, we will meet special cases where QR is the method of choice.

The standard algorithm for the QR decomposition involves successive Householder transformations (to be discussed later in §11.3). We write a Householder matrix in the form $\mathbf{1} - \mathbf{u} \otimes \mathbf{u}/c$, where $c = \frac{1}{2} \mathbf{u} \cdot \mathbf{u}$. An appropriate Householder matrix applied to a given matrix can zero all elements in a column of the matrix situated below a chosen element. Thus we arrange for the first Householder matrix $\mathbf{Q}_0$ to zero all elements in column 0 of $\mathbf{A}$ below the zeroth element. Similarly, $\mathbf{Q}_1$ zeroes all elements in column 1 below element 1, and so on up to $\mathbf{Q}_{n-2}$. Thus

$$\mathbf{R} = \mathbf{Q}_{n-2} \cdots \mathbf{Q}_0 \cdot \mathbf{A} \quad (2.10.5)$$

Since the Householder matrices are orthogonal,

$$\mathbf{Q} = (\mathbf{Q}_{n-2} \cdots \mathbf{Q}_0)^{-1} = \mathbf{Q}_0 \cdots \mathbf{Q}_{n-2} \quad (2.10.6)$$

In many applications $\mathbf{Q}$ is not needed explicitly, and it is sufficient to store only the factored form (2.10.6). (We do, however, store $\mathbf{Q}$, or rather its transpose, in the code below.) Pivoting is not usually necessary unless the matrix $\mathbf{A}$ is very close to singular. A general QR algorithm for rectangular matrices including pivoting is given in [1]. For square matrices and without pivoting, an implementation is as follows:

```
struct QRdcmp {
```

Object for QR decomposition of a matrix $\mathbf{A}$, and related functions.

```
    Int n;
```

```
    MatDoub qt, r;
```

```
    Bool sing;
```

```
    QRdcmp(MatDoub_I &a);
```

```
    void solve(VecDoub_I &b, VecDoub_O &x);
```

```
    void qtmult(VecDoub_I &b, VecDoub_O &x);
```

```
    void rsolve(VecDoub_I &b, VecDoub_O &x);
```

```
    void update(VecDoub_I &u, VecDoub_I &v);
```

```
    void rotate(const Int i, const Doub a, const Doub b);
```

```
};
```

Stored $\mathbf{Q}^T$ and $\mathbf{R}$.

Indicates whether $\mathbf{A}$ is singular.

Constructor from $\mathbf{A}$.

Solve $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.

Multiply $\mathbf{Q}^T \cdot \mathbf{b} = \mathbf{x}$.

Solve $\mathbf{R} \cdot \mathbf{x} = \mathbf{b}$ for $\mathbf{x}$.

See next subsection.

Used by update.

qrncmp.h

As usual, the constructor performs the actual decomposition:

```
QRdcmp::QRdcmp(MatDoub_I &a)
```

```
    : n(a.nrows()), qt(n,n), r(a), sing(false) {
```

Construct the QR decomposition of $\mathbf{a}[0..n-1][0..n-1]$. The upper triangular matrix $\mathbf{R}$ and the transpose of the orthogonal matrix $\mathbf{Q}$ are stored. `sing` is set to true if a singularity is encountered during the decomposition, but the decomposition is still completed in this case; otherwise it is set to false.

```
    Int i,j,k;
```

```
    VecDoub c(n), d(n);
```

```
    Doub scale,sigma,sum,tau;
```

qrncmp.h

```

for (k=0;k<n-1;k++) {
    scale=0.0;
    for (i=k;i<n;i++) scale=MAX(scale,abs(r[i][k]));
    if (scale == 0.0) {
        sing=true;
        c[k]=d[k]=0.0;
    } else {
        Form  $\mathbf{Q}_k$  and  $\mathbf{Q}_k \cdot \mathbf{A}$ .
        for (i=k;i<n;i++) r[i][k] /= scale;
        for (sum=0.0,i=k;i<n;i++) sum += SQR(r[i][k]);
        sigma=SIGN(sqrt(sum),r[k][k]);
        r[k][k] += sigma;
        c[k]=sigma*r[k][k];
        d[k] = -scale*sigma;
        for (j=k+1;j<n;j++) {
            for (sum=0.0,i=k;i<n;i++) sum += r[i][k]*r[i][j];
            tau=sum/c[k];
            for (i=k;i<n;i++) r[i][j] -= tau*r[i][k];
        }
    }
}
d[n-1]=r[n-1][n-1];
if (d[n-1] == 0.0) sing=true;
for (i=0;i<n;i++) {
    for (j=0;j<n;j++) qt[i][j]=0.0;
    qt[i][i]=1.0;
}
for (k=0;k<n-1;k++) {
    if (c[k] != 0.0) {
        for (j=0;j<n;j++) {
            sum=0.0;
            for (i=k;i<n;i++)
                sum += r[i][k]*qt[i][j];
            sum /= c[k];
            for (i=k;i<n;i++)
                qt[i][j] -= sum*r[i][k];
        }
    }
}
for (i=0;i<n;i++) {
    r[i][i]=d[i];
    for (j=0;j<i;j++) r[i][j]=0.0;
}
}

```

Form $\mathbf{Q}^T$ explicitly.

Form $\mathbf{R}$ explicitly.

The next set of member functions is used to solve linear systems. In many applications only the part (2.10.4) of the algorithm is needed, so we put in separate routines the multiplication $\mathbf{Q}^T \cdot \mathbf{b}$ and the backsubstitution on $\mathbf{R}$.

```

qrncmp.h void QRncmp::solve(VecDoub_I &b, VecDoub_O &x) {
    Solve the set of n linear equations  $\mathbf{A} \cdot \mathbf{x} = \mathbf{b}$ .  $\mathbf{b}[0..n-1]$  is input as the right-hand side vector,
    and  $\mathbf{x}[0..n-1]$  is returned as the solution vector.
    qtmult(b,x);          Form  $\mathbf{Q}^T \cdot \mathbf{b}$ .
    rsolve(x,x);          Solve  $\mathbf{R} \cdot \mathbf{x} = \mathbf{Q}^T \cdot \mathbf{b}$ .
}

void QRncmp::qtmult(VecDoub_I &b, VecDoub_O &x) {
    Multiply  $\mathbf{Q}^T \cdot \mathbf{b}$  and put the result in  $\mathbf{x}$ . Since  $\mathbf{Q}$  is orthogonal, this is equivalent to solving
     $\mathbf{Q} \cdot \mathbf{x} = \mathbf{b}$  for  $\mathbf{x}$ .
    Int i,j;
    Doub sum;
    for (i=0;i<n;i++) {
        sum = 0.;

```

```

        for (j=0;j<n;j++) sum += qt[i][j]*b[j];
        x[i] = sum;
    }
}

```

```
void QRdcmp::rsolve(VecDoub_I &b, VecDoub_O &x) {
```

Solve the triangular set of n linear equations $\mathbf{R} \cdot \mathbf{x} = \mathbf{b}$. $\mathbf{b}[0..n-1]$ is input as the right-hand side vector, and $\mathbf{x}[0..n-1]$ is returned as the solution vector.

```

    Int i,j;
    Doub sum;
    if (sing) throw("attempting solve in a singular QR");
    for (i=n-1;i>=0;i--) {
        sum=b[i];
        for (j=i+1;j<n;j++) sum -= r[i][j]*x[j];
        x[i]=sum/r[i][i];
    }
}

```

See [2] for details on how to use QR decomposition for constructing orthogonal bases, and for solving least-squares problems. (We prefer to use SVD, §2.6, for these purposes, because of its greater diagnostic capability in pathological cases.)

2.10.1 Updating a QR decomposition

Some numerical algorithms involve solving a succession of linear systems each of which differs only slightly from its predecessor. Instead of doing $O(N^3)$ operations each time to solve the equations from scratch, one can often update a matrix factorization in $O(N^2)$ operations and use the new factorization to solve the next set of linear equations. The LU decomposition is complicated to update because of pivoting. However, QR turns out to be quite simple for a very common kind of update,

$$\mathbf{A} \rightarrow \mathbf{A} + \mathbf{s} \otimes \mathbf{t} \quad (2.10.7)$$

(compare equation 2.7.1). In practice it is more convenient to work with the equivalent form

$$\mathbf{A} = \mathbf{Q} \cdot \mathbf{R} \rightarrow \mathbf{A}' = \mathbf{Q}' \cdot \mathbf{R}' = \mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v}) \quad (2.10.8)$$

One can go back and forth between equations (2.10.7) and (2.10.8) using the fact that $\mathbf{Q}$ is orthogonal, giving

$$\mathbf{t} = \mathbf{v} \quad \text{and either} \quad \mathbf{s} = \mathbf{Q} \cdot \mathbf{u} \quad \text{or} \quad \mathbf{u} = \mathbf{Q}^T \cdot \mathbf{s} \quad (2.10.9)$$

The algorithm [2] has two phases. In the first we apply $N - 1$ Jacobi rotations (§11.1) to reduce $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg form. Another $N - 1$ Jacobi rotations transform this upper Hessenberg matrix to the new upper triangular matrix $\mathbf{R}'$. The matrix $\mathbf{Q}'$ is simply the product of $\mathbf{Q}$ with the $2(N - 1)$ Jacobi rotations. In applications we usually want $\mathbf{Q}^T$, so the algorithm is arranged to work with this matrix (which is stored in the `QRdcmp` object) instead of with $\mathbf{Q}$.

```
void QRdcmp::update(VecDoub_I &u, VecDoub_I &v) {
```

Starting from the stored QR decomposition $\mathbf{A} = \mathbf{Q} \cdot \mathbf{R}$, update it to be the QR decomposition of the matrix $\mathbf{Q} \cdot (\mathbf{R} + \mathbf{u} \otimes \mathbf{v})$. Input quantities are $\mathbf{u}[0..n-1]$, and $\mathbf{v}[0..n-1]$.

```

    Int i,k;
    VecDoub w(u);
    for (k=n-1;k>=0;k--)
        if (w[k] != 0.0) break;
    if (k < 0) k=0;
    for (i=k-1;i>=0;i--) {
        rotate(i,w[i],-w[i+1]);
        if (w[i] == 0.0)
            w[i]=abs(w[i+1]);
    }
}

```

Find largest k such that $u[k] \neq 0$.

Transform $\mathbf{R} + \mathbf{u} \otimes \mathbf{v}$ to upper Hessenberg.

`qrncmp.h`